

CGPS CASNet centerline biomarker plan

2026-09-27

Running frozen CAS-Net safely on CGPS

CAS-Net stays exactly as trained: the plan loads the bare `cas_net.pt` state dict strictly, through the same `inference.py` path already used for ImageCAS-X, and never touches a gradient again once it points at Herlev-Østerbro (CGPS) scans instead. Centerlines then come from a mask-to-skeleton step, `topology/skeleton.py`, that exists only on the project’s `week3` branch and is not runnable today because two of its providers assume ground truth that CGPS does not have; recovering and de-oracling that file is the single largest piece of new engineering this plan requires. Tortuosity is reported as one locked pair of descriptors, mean absolute curvature from chord turning angles (κ_a) as primary and arc/chord (TI) as secondary, computed with no additional smoothing beyond the 0.5 mm/5-vertex smoothing already built into centerline construction, because the project’s own descriptor audits show that adding smoothing on top of that buys reliability only by silently erasing real curvature signal. Age and sex enter a linear mixed-effects model with a per-patient random intercept and per-vessel residual, a structure that extends to any further CGPS biomarker column by the same fixed-effect-plus-multiplicity-correction rule rather than a bespoke model per variable. Because CGPS has no manual coronary segmentation, the entire quality-control layer has to run without a Dice score or an HD95 anywhere in it, which is unusual for this kind of deployment but is explicitly the gap the thesis is positioned to address rather than paper over. Every number this plan will eventually produce, CGPS reference ranges, effect sizes for age and sex, is a first measurement, not a replication of an existing figure, and the plan is written to say so plainly rather than to imply otherwise.

Inference itself needs no new code, only a CGPS-pointed config

Running CAS-Net on CGPS requires zero changes to `inference.py`. The entry point `predict()` builds the preprocessing pipeline, the postprocessing pipeline, and a test loader, then loads the model with `BaseLumenModel.load_weights`, which resolves the checkpoint to `<run_dir>/<method>_best.pt` and performs a strict `load_state_dict` (`inference.py:398-425`). For the delivered pretrained run that resolves to `cas_net_pretrained/cas_net_best.pt`, a symlink to `$ImageCAS_X_weights_path/cas_net.pt`, the identical frozen ImageCAS-X-trained weights this plan keeps using on CGPS. Per scan, `run_inference()` builds the four-way mirror test-time-augmentation combo list over the X and Y tensor axes only (`inference.py:127-135`), dispatches to `patch_stitch_inference()` for CAS-Net’s `random_crop` input type, which tiles the volume into $128 \times 160 \times 160$ patches at a fixed stride of `round(patch_size \times (1 - overlap))`, giving strides of 64, 80, 80 voxels at the configured 50 percent overlap (`inference.py:70-86`), and blends overlapping logits with an unnormalised Gaussian weight of scale $\sigma = \text{patch_size} \times 0.125$ (nnU-Net’s convention), dividing the accumulated logits by the accumulated weight at the end so the

kernel never needs pre-normalising (`inference.py:58-67`, 118-121). Sigmoid is applied before any resampling, deliberately, so a continuous probability rather than an already-binarised mask survives the change of resolution (`inference.py:296-297`, 500); the two-channel probability map is then resampled with `sitkLinear` onto the original scan's own geometry (`inference.py:294-312`), and only then does `ArgmaxBinarize` followed by `KeepComponentsLargerThan100Voxels` (26-connected, `min_size=100`) produce the final mask (`postprocessing/steps.py:23-55`). None of this reads or needs a ground-truth mask; the only inputs are the volume itself and the frozen weights.

What CGPS actually changes is entirely on the data side of the config, and none of it exists yet: **no CGPS-specific inference config is committed anywhere in the repository**, so `data.volume_dir`, `data.volume_suffix`, and whichever resampled-cache path the config's `preprocessing.steps` implies all need to be written fresh, pointed at wherever the CGPS scans and their metadata table live. `evaluate.py`'s own docstring states it never loads a model and can score externally produced predictions dropped into a `predictions/` directory (`inference.py:11-12`), which is the intended mechanism for an inference-only cohort; for CGPS the corollary is that `evaluate.py` simply should not be run at all, since every metric in `configs/pipeline.json`'s `evaluation.metrics` list takes a ground-truth mask or centerline as a hard argument (`utils/metrics.py`, e.g. `hausdorff_95` raises `ValueError` on an empty ground truth at `utils/metrics.py:56-57`). This is not a workaround; it is the documented, supported mode of the tool. The one deviation to record here explicitly is that the checkpoint must never be a CGPS-adapted file: CLAUDE.md's own gotcha exists precisely because `train.py` and `inference.py` share `build_model`, so a stray checkpoint path silently turns a "frozen inference" run into an accidental fine-tune, and this plan's entire premise, that CGPS numbers describe an unmodified ImageCAS-X model, depends on that never happening.

The two 0.5 mm operations in this repository are unrelated and must not be conflated

Both because CGPS planning language tends to say "the same 0.5 mm smoothing already used in this repo" loosely, and because the task explicitly requires the distinction be made precise, it is worth stating exactly what each of the two 0.5 mm quantities is a property of.

The first is a property of a dense 3-D scalar grid, not of any curve. `preprocessing/steps.py`'s `Resample` class resamples the `sitk` volume and mask to `target_spacing = 0.5` mm isotropic, using `sitkLinear` interpolation for the image and `sitkNearestNeighbor` for the mask (`preprocessing/steps.py:6-49`; `configs/pipeline.json`). This is run once, offline, by `utils/offline_resample_images_to_disk.py`, and cached as `.npy` arrays under `volumes_resampled/` and `segmentations_resampled/`. At training and inference time, `base_dataset.py:_load_volume` reads straight from that cache with `np.load(..., mmap_mode="r")` whenever "resample" appears in `preprocessing.steps`, and `_run_preprocessing` explicitly skips the `Resample` step class itself at runtime in that mode (`dataloading/base_dataset.py:29`, 127-146); the resample is baked into the cache, never repeated per sample. This is the geometry the CNN reads over; it has nothing to do with any curve and it applies to CGPS exactly as it applies to ImageCAS-X, modulo needing a fresh resample-to-disk pass for whatever native CGPS spacing the scans arrive at, since `volumes_resampled/` today only covers ImageCAS-X's 800 curated scans.

The second is a property of a sparse ordered sequence of points, produced only after a mask has already been reduced to a 1-D curve. `topology/skeleton.py`'s module docstring states that

the delivered ground-truth centerlines are “a skeleton of the corrected mask, Gaussian smoothed (sigma = 0.5 mm, 5-vertex window)... (Bransby et al. 2026, Methods),” and its `smooth()` function reproduces that same operation on a predicted mask’s skeleton. Concretely, for a polyline’s interior points $p_1, \dots, p_n$ with arc-length positions $\text{arc}_1, \dots, \text{arc}_n$, each interior point is replaced by a Gaussian-weighted average over a local window of the nearest $w = 5$ vertices measured along arc length:

$$p'_j = \frac{\sum_{k \in \text{window}} w_{jk} p_k}{\sum_{k \in \text{window}} w_{jk}}, \quad w_{jk} = \exp\left(-\frac{1}{2} \left(\frac{\text{arc}_k - \text{arc}_j}{\sigma_{\text{mm}}}\right)^2\right), \quad \sigma_{\text{mm}} = 0.5.$$

Junction and end points are deliberately left unmoved, since the loop only runs over interior points (`topology/skeleton.py:53-64`). This operates on a handful of 3-D coordinates along an already-extracted curve, a completely different array, operation, and purpose from the voxel-grid resample above; the two literals, `target_spacing=0.5` in `configs/pipeline.json` and `sigma_mm=0.5` in `topology/skeleton.py`, are independent constants in unrelated files that happen to share a numeral, and no code path in the repository derives one from the other. A third, still different smoothing exists downstream of this one, inside `topology/tortuosity.py`, with its own defaults (`RESAMPLE_MM = 0.25`, `DEFAULT_SIGMA_MM = 1.0`), applied to an arc-length-resampled point set as a preprocessing step before curvature/torsion computation; it should likewise not be confused with either of the two above, and its module is not what this plan uses for descriptor computation (see below). For this plan, “the 0.5 mm smoothing already used in this repo” refers unambiguously to `topology/skeleton.py:smooth()`’s arc-length Gaussian pass on skeleton coordinates, and nothing else in the codebase currently implements a code-verified 0.5 mm smoothing operation on a centerline.

From a binary mask to a labelled tree: what exists, what has to be recovered, and what has to be built new

Two structurally separate jobs sit between a CAS-Net prediction mask and a usable centerline tree, and only one of them is present on the branch this plan runs from. `topology/graph.py:build_tree()` (present on the current branch) takes an already-loaded `io.Centerline` object, i.e. VTK points, polylines, and per-point start/end/branch flags that already exist, computes point degree from polyline endpoints, identifies junctions, merges degree-2 pass-through points into maximal junction-to-junction chains, finds connected components, roots each component at an existing start/ostium point or the point nearest the rest of the tree, and does a breadth-first traversal orienting every chain proximal to distal into named `Segment` objects, dropping cycle-closing edges into `chords` (`topology/graph.py:367-484`). It performs no skeletonisation of its own; it needs a VTK file with topology markers already attached before it can run.

Producing that VTK file from a raw binary mask is exactly what `topology/skeleton.py` does, and it is **not present on the current branch** (`tree-extraction_essential`); it exists only on `week3`, removed by commit `4aab47a` (“Trim to tree-building + angle/tortuosity feature extraction”), whose own message names it, `scripts/build_predicted_centerlines.py`, and `scripts/validate_extraction.py` as “still an open problem... not part of this objective.” Its pipeline, once recovered, crops the mask to its bounding box, runs `skimage.morphology.skeletonize` (Lee et al. 1994 3-D binary thinning), traces ordered polylines through `skan.Skeleton`, converts voxel indices to physical LPS millimetres, applies the 0.5 mm smoothing above, and then does two things that are the actual blocker for CGPS use: it assigns each skeleton point a left/right/vessel label by nearest-neighbour lookup against the ground-truth mask’s own labelled

voxels (`oracle_labels()`), and it finds each side’s ostium by matching a predicted degree-1 endpoint to the ground-truth ostium location within `OSTIUM_MAX_MM = 10.0` mm. Both are called “oracle” in the module’s own docstring, and the docstring states outright that “Herlev-Østerbro needs real [providers],” meaning a real aorta localisation and a real multi-class vessel labeller, neither of which exists anywhere in this project’s git history across every branch checked (`main`, `tree-extraction`, `week1` through `week3`, `clean_version`, `thesis-setup`, `tortuosity_focus`, `tree-extraction_essential`).

This plan replaces both oracle providers with deterministic, non-learned substitutes rather than deferring the problem, because CGPS predictions will not have a ground-truth mask to consult and the mask-to-centerline step has to run regardless. For left/right splitting, the substitute uses the fact that CAS-Net’s postprocessed mask is designed to leave behind exactly two anatomically separate connected components, one per coronary tree, whenever segmentation succeeds (`KeepComponentsLargerThan100Voxels` is size-based specifically so both trees survive as separate components, `postprocessing/steps.py:25-26`); each connected component is therefore its own side, with no voxel-level label lookup needed at all. For ostium placement, the substitute needs a coarse anatomical anchor in place of the ground-truth start point; the most direct route available without new ground truth is a coarse aorta/aortic-root localisation, for which this plan uses `TotalSegmentator`, an off-the-shelf tool already referenced in this project’s own walkthrough documentation for a different purpose, to produce a rough aortic-root region per CGPS scan, and takes each component’s degree-1 endpoint nearest that region as its ostium. Neither substitute has ever been validated against anything, on CGPS or on ImageCAS-X, and that absence of validation is itself a named risk this plan carries forward rather than resolves: the component-based left/right split is spatially unambiguous but silently wrong if a scan has more than two well-formed trees (see below), and the aorta-proximity ostium heuristic is a design choice, not a measured one. The mitigation available without ground truth is visual spot-checking through `utils/display_in_slicer.py`’s existing “`segments`” mode, which already loads a mask plus left/right centerline VTKs with no ground-truth comparison built in (`utils/display_in_slicer.py:102-131`) and needs only its `MASK_DIR/CENTERLINES_DIR` constants repointed at the CGPS prediction and derived-centerline directories, no new code, to serve as a manual review tool for exactly this substitution.

A separate, independent skeletoniser, `postprocessing/qiu/skeleton.py`, exists only to support fragment-reconnection candidate-finding on the raw 0.5 mm voxel grid and is not part of the final centerline product; it is discussed below only because the reconnection code it feeds is directly relevant to CGPS robustness.

The plan uses “keep2,” not the learned Qiu reconnection, and states exactly why

CAS-Net’s postprocessing already keeps every mask component above 100 voxels, which on a genuinely out-of-distribution scan can leave more than two components: an OOD scan producing more noise-driven false-positive fragments than a typical ImageCAS-X scan will retain more spurious small components than the two anatomically expected trees, degrading the tree structure the skeleton and graph steps then have to work with. Two candidate cleanup policies were built and calibrated on ImageCAS-X’s own train/val/test split before being deliberately removed from the current branch: a full Qiu-et-al.-2025-style reconnection classifier (“`qiu`” mode) that scores each fragment’s join candidates with a trained trust classifier P and a threshold-based accept/reject rule, and a deterministic “keep2” policy (`ReconnectParams(mode="keep_largest", n_main_components=2)`) that simply discards every component except the two largest by size. Both live only on `week3`, in `postprocessing/qiu/reconnect.py` and its siblings.

On ImageCAS-X’s own 160-scan test split, `qiu` was in fact the more cost-efficient of the two: it matched `keep2`’s best Betti-error-1 improvement (-1.831 , a tie) while costing less on every other tracked metric (dice -0.005 vs. -0.012 , `cl_dice` -0.006 vs. -0.017 , `centerline_hd95` $+1.98$ mm vs. $+4.64$ mm), because the reconnection walk tends to reattach genuinely real fragments (53.9 percent of the 293 removed test fragments carried ground-truth overlap ≥ 0.5) using the join geometry itself, not the classifier’s confidence. But that same classifier’s actual per-fragment discrimination was weak even in-distribution: `rec_acc` of only 0.502 against a base rate of 0.461 from deleting everything, despite a validation AUC of 0.977, “barely above the base rate despite val AUC 0.977” in the project’s own recorded conclusion, and the accept/reject threshold $T = 1.0$ with ADF disabled was calibrated exclusively on ImageCAS-X’s own validation split, a calibration that cannot be repeated on CGPS because CGPS has no ground truth to calibrate against.

This plan locks in **keep2**, not **qiu**, for the CGPS deployment, on a risk basis rather than a raw-cost basis: `keep2`’s only failure mode is discarding a real small distal branch, a bounded, predictable, and directly inspectable error that never invents connectivity that was not there; `qiu`’s failure mode is a frozen, already-weak-in-distribution learned threshold operating on a further, unmeasured out-of-distribution shift with no ground truth anywhere to reveal whether it is silently reattaching spurious fragments as if they were real vessel. A classifier whose in-distribution discrimination is close to chance is not a safe object to extrapolate past a distribution shift the project cannot even quantify. Recovering `keep2` is also the lighter engineering lift: only `postprocessing/qiu/reconnect.py`’s `keep_largest` mode is needed, with no classifier training and no walk/ADF machinery, since `keep2` requires none of `classifier.py` or `walk.py`.

One locked descriptor pair, no additional smoothing, and an explicit exception for the left main

Nine internal reports converge, after real churn, on a single non-blended pair of whole-vessel tortuosity descriptors, and this plan adopts exactly that pair rather than any composite or any of the several descriptors the reports evaluated and rejected. The primary is mean absolute curvature from chord turning angles, κ_a : resample the centerline at a fixed chord length ℓ to obtain unit chord directions $u_1, \dots, u_n$, compute the turning angle between consecutive directions, $\theta_j = \arccos(u_j \cdot u_{j+1})$, and define

$$\kappa_a = \frac{\sum_j \theta_j}{L} \quad [\text{mm}^{-1}], \quad K = \kappa_a \cdot L = \sum_j \theta_j \quad (\text{total turning, radians, additive across segments}).$$

The secondary is arc/chord, $DM = L/D$ and $TI = DM - 1 = L/D - 1$, where L is the smoothed arc length and D the Euclidean end-to-end chord distance; an exact identity connects the two via the mean unit tangent $\bar{T}$ and tangent variance $V = 1 - |\bar{T}|^2$, namely $D = L|\bar{T}|$, so $L/D = (1 - V)^{-1/2}$ and $TI \approx V/2$ for small V , which is the formal reason TI is dominated by long, gentle bends and vessel length rather than local bending (Spearman $\rho(DM, \text{length})$ of 0.69 to 0.84 across LAD/LCx/RCA on the ImageCAS-X training split, versus 0.20 to 0.32 for κ_a). A third candidate, $Q = (\kappa_r/\kappa_a)^2$ built from the curvature second moment, was proposed as a co-primary and then dropped outright: it is non-monotone even on noise-free synthetic curves, its cohort median (1.28 to 1.43) is statistically indistinguishable from its value under pure measurement noise ($4/\pi \approx 1.27$), and it shows essentially no within-patient coherence across a patient’s own vessels ($|\rho| \leq 0.16$), unlike κ_a ’s ρ up to 0.60. This plan does not compute Q .

The single most consequential design decision this plan locks in is **no additional smoothing beyond the 0.5 mm/5-vertex smoothing already baked into centerline construction**,

i.e. an extra-smoothing parameter of $\sigma = 0$ at the descriptor-computation stage, using the delivered/extracted centerline coordinates exactly as `topology/skeleton.py` produces them. This is not the only position taken across the nine reports; the later “vessel specific” report recommends an extra $\sigma = 1$ mm pass on top, and the readiness audit names this disagreement explicitly as unresolved and blocking. The plan resolves it in favour of $\sigma = 0$ for three concrete reasons, each derived quantitatively elsewhere in the same corpus: first, over-smoothing directly deflates the two quantities being measured, shrinking arc length by approximately $\frac{w^2}{2}\kappa_r^2$ and, at a fixed chord ℓ , aliasing κ_a down by as much as 98 percent on a 10 mm-wavelength bend, a far larger effect than any noise it removes; second, reliability (ICC) was shown to rise monotonically with smoothing width under every jitter model tested, meaning any rule that simply chases higher reliability will always select the most aggressive smoothing on offer and silently trade away real signal for apparent stability, an explicitly named degenerate selection rule in the descriptor audits; third, the case study’s own direct measurement found κ_a ’s cohort-percentile *ranking* barely moved between $\sigma = 0$ and $\sigma = 1$ mm (moving from the 10.5th to 9.1st percentile and from the 88.2nd to 87.9th percentile on the two vessels tested), so there is little discriminative power to gain by adding smoothing in the first place. Keeping $\sigma = 0$ also keeps the CGPS numbers on the same operating point as whatever development-distribution numbers already exist for ImageCAS-X, which matters directly if this thesis ever compares the two cohorts. The fixed chord is set to $\ell = 5$ mm, falling back to 8 mm only where a segment is too short to support a meaningful number of 5 mm chords; four independent scale-selection arguments in the descriptor corpus (a noise floor under 10 percent of signal, cancellation of the two opposing length biases, an MSE-optimal smoothing width, and resolution of the tightest physiologically plausible coronary bend near $R \approx 3$ mm at 90 degrees) all land between roughly 2 and 5 mm, which is the basis for this default.

The left main is an explicit, named exception, not a silently dropped row: its median length of roughly 7 to 10.5 mm leaves κ_a undefined in 72 percent of cases at a 5 mm chord, so this plan reports the left main by length and by a bifurcation angle instead of a tortuosity value, writing an explicit NaN with a stated reason wherever κ_a cannot be computed. That bifurcation angle uses the `fitcore3` definition already vetted for regional descriptors: at a junction with radius r_J , fit a straight line to each daughter branch’s centerline points over the arc-length window $[r_J, r_J + 3 \text{ mm}]$, and take the angle between the two fitted daughter directions, $\angle = \arccos(d_1 \cdot d_2)$; this proved far more stable across smoothing choices (78.0 to 80.8 degrees) than a naive secant-endpoint tangent, because a chord-endpoint position error σ over a chord of length d propagates to an angular error of approximately $\sqrt{2}\sigma/d$, and a longer fitted window suppresses that error directly. Alongside it, the Finet ratio $r_0/(r_1 + r_2)$ (parent radius over the sum of the two daughter radii, measured with a mask Euclidean-distance-transform radius over the same window) is reliable specifically at the left main (ICC 0.85 to 0.90). If regional descriptors are computed at all beyond the left main, one concrete, already-identified bug must be fixed first: the regional κ_a code in `tortuosity/research/case_study/regional.py` divides attributed turning angle by the full region length even though the first half-chord, about 2.5 mm at $\ell = 5$ mm, of any window never receives an attributed turning angle at all, biasing regional κ_a downward by roughly $\frac{\ell/2}{W}$, about 14 percent at an 18 mm window and 25 percent at a 10 mm floor, and reintroducing exactly the length-dependence κ_a was chosen to avoid; the fix is to start every proximal window at 5 mm from the junction, matching the convention the RCA window already uses, or to divide by $\ell/2$ times the number of attributed vertices rather than the raw window length. This plan does not compute regional descriptors until that fix is applied.

Two engineering loose ends sit underneath every number this section produces. The committed

`scc_density(P, chord_len)` function in `tortuosity/merged.py` computes $\sum_j \theta_j / (\pi \ell(n-1)) = \sum_j \theta_j / (\pi L)$, which differs from the $\kappa_a = \sum_j \theta_j / L$ convention this plan (and every cited normal-range number) uses by a factor of π ; any CGPS number computed by calling that function directly must be multiplied by π first. And the vessel definition itself is ambiguous in code: `tortuosity/vessels.py` defines a named vessel as the longest geodesic path through same-labelled points, while `topology.graph.Vessel` chains same-named segments and can terminate at a different point; the two are not interchangeable, and this plan fixes the choice to `vessels.py`'s longest-geodesic-path definition, recorded here as a locked decision rather than left open, because it is the definition every cited ImageCAS-X normal-range number in this corpus was actually computed under.

A single mixed-effects model that extends to any biomarker CGPS supplies

The unit of observation is one descriptor value per vessel per patient, and vessels from the same patient share the same age, sex, scanner, disease burden, and image quality, so their residuals are correlated rather than independent; a plain per-vessel ordinary least squares regression treats every vessel-row as an independent draw, understating standard errors and inflating the false-positive rate on exactly the age and sex test this plan cares about ([Coursera: What Are Linear Mixed Effects Models?](#); [Trials: mixed model for repeated measures simulation study](#)). A linear mixed-effects model (LMM) is the standard remedy, adding a patient-level random intercept alongside the fixed effects of interest, and an applied vascular-geometry precedent uses exactly this structure to relate geometric features to outcomes while a random effect absorbs variability between vascular trees ([Vascular Geometry Characterization for AI-Based Endovascular Navigation](#)). The base model, indexing patient by i and vessel by j , is

$$y_{ij} = \beta_0 + \beta_1 \text{age}_i + \beta_2 \text{sex}_i + \beta_3 \text{vesselType}_{ij} + u_i + \varepsilon_{ij}, \quad u_i \sim N(0, \tau^2), \quad \varepsilon_{ij} \sim N(0, \sigma^2),$$

with u_i and ε_{ij} independent, u_i capturing a patient's overall vascular tendency (global tortuosity, calcification burden, image quality of that particular scan) shared across all of that patient's vessels, and ε_{ij} the vessel-level residual. The intraclass correlation $ICC = \tau^2 / (\tau^2 + \sigma^2)$ is reported alongside the model as a diagnostic in its own right: a near-zero ICC would say clustering barely matters here, while an ICC above roughly 0.1 to 0.2 confirms the naive-OLS concern was real for this specific outcome.

Sex enters as a single dummy variable; age is not assumed linear by default, and is first tested with a quadratic or restricted-cubic-spline term against the linear-only model by a likelihood-ratio test before the linear form is accepted, since coronary tortuosity's relationship with age is not established to be linear in this cohort or any other reviewed. An age-by-sex interaction is the correct way to test whether a sex effect is itself age-dependent (motivated by one cohort reporting women's tortuosity prevalence rising specifically from around age 45 onward), rather than stratifying by sex, which discards shared variance information and loses power. Body surface area and coronary dominance are added as covariates when the outcome is length-scale sensitive; BSA-adjustment is documented to remove an otherwise-significant sex difference in absolute coronary diameter (roughly 9 percent smaller in women unadjusted) ([PMC10811496](#); [Circulation 1992](#)), and dominance materially affects proximal RCA and LCx caliber though not LAD ([PMC6584238](#)); because κ_a and TI are already ratios rather than absolute lengths, whether BSA meaningfully changes their association with age and sex is treated as an open sensitivity check (fit with and without BSA) rather than an assumed necessity, since no source validated BSA-adjustment specifically for a tortuosity ratio as opposed

to an absolute caliber measurement. The extended specification is

$$y_{ij} = \beta_0 + \beta_1 \text{age}_i + \beta_2 \text{age}_i^2 + \beta_3 \text{sex}_i + \beta_4 \text{vesselType}_{ij} + \beta_5 \text{BSA}_i + \beta_6 \text{dominance}_i + u_i + \varepsilon_{ij}.$$

CGPS is expected to carry biomarker columns beyond age and sex, and rather than redesign the model for each one as it appears, this plan generalises by appending one covariate at a time to the same fixed model class: for each additional biomarker b_k , $k = 1, \dots, K$, fit

$$y_{ij} = \beta_0 + \beta_1 \text{age}_i + \beta_2 \text{sex}_i + \beta_3 \text{vesselType}_{ij} + \gamma_k b_{k,i} + u_i + \varepsilon_{ij},$$

centering and scaling continuous b_k and dummy-coding categorical b_k , always retaining age and sex as covariates. Age and sex are the single pre-registered, confirmatory test; the resulting panel of K estimates $\hat{\gamma}_k$ is treated as a secondary, hypothesis-generating panel and corrected for multiple comparisons with Benjamini-Hochberg false-discovery-rate control at $q = 0.05$, reporting each effect with its 95 percent confidence interval rather than a bare significance flag. This one-covariate-at-a-time structure is what lets the plan generalise to “whatever additional biomarker columns CGPS provides”: no biomarker requires a new model family, only one more γ_k fit inside the loop and one more term in the multiplicity correction, and it avoids the collinearity and degrees-of-freedom problems of folding an unknown number of covariates into a single omnibus model before knowing how many there are or how correlated they are with each other.

No coronary-tortuosity-specific closed-form power formula for this design exists in the literature; general LMM power and sample-size methods are documented to be sparse for exactly this reason, and the standard modern route is simulation-based power analysis fit on estimated variance components ([PMC7009022](#); [PMC8613146](#); [simr tutorial](#)). Because CGPS’s cohort size is fixed rather than a design choice, the plan reports a minimum detectable effect at 80 percent power for the achieved sample rather than a target sample size, using $\hat{\tau}^2$ and $\hat{\sigma}^2$ estimated from CGPS’s own fitted model, and uses the design-effect approximation $n_{\text{required}} \approx n_{\text{naive}} \times (1 + (m - 1) \cdot ICC)$, where m is the mean number of vessels per patient, as a first-pass sanity check before running the full simulation. It is important to state plainly what does not exist yet rather than imply it does: **no CGPS-specific normal range or previously calibrated effect size for age or sex on coronary tortuosity currently exists anywhere**, on this project or, as far as the reviewed literature shows, for a continuous curvature-based descriptor in any cohort; a 45.2 percent versus 19.7 percent sex-prevalence contrast in one angiographic cohort ([PubMed 28616559](#)) is a categorical prevalence figure, not a standardised effect size for a continuous κ_a , and its full text could not be verified in this pass, so it is cited here only as illustrative context, not a number to plug into a power calculation. Similarly, a much larger 22,334-patient multivariable study reportedly found no independent age effect once other risk factors were adjusted for, while sex remained associated ([PMC13308244](#)), but that paper’s full text was blocked by a bot-check page in this research pass and should be re-verified directly before its adjustment set or effect direction is cited with a specific number in the thesis. This analysis is therefore positioned, and should be written up, as establishing the first CGPS-specific age/sex reference distribution for these descriptors, not as replicating or contradicting an existing one. For the reference-range component of that first measurement specifically, the same CLSI EP28-A3c nonparametric methodology already recommended for ImageCAS-X applies unchanged: 2.5th/97.5th percentile intervals with 90 percent binomial confidence intervals, requiring at least roughly 120 cases per partition, dominance entered as a covariate rather than a stratum given the small number of left- and co-dominant cases typically available, and length- and dominance-conditional centiles fit by quantile regression or a GAMLSS/LMS model with a Box-Cox transform for κ_a and TI respectively.

Out-of-distribution by construction, and what QC is possible with no ground truth at all

CGPS differs from ImageCAS-X in scanner, acquisition protocol, and referral population, and the model deployed on it is never retrained or fine-tuned on a single CGPS voxel; this plan treats that gap as a stated, load-bearing premise rather than an incidental caveat. The literature on cross-cohort coronary-CT segmentation deployment reports exactly this kind of drop: one coronary-CT generalisation study found Dice fall from roughly 0.90 to 0.80 moving between cohorts, with the largest gap in low-calcification cases, and found artery contrast enhancement ($r = 0.408$) and edge sharpness ($r = 0.239$) both correlated positively with the resulting accuracy, implicating scanner and reconstruction protocol differences directly (PMC13230296). Systematic out-of-distribution testing is also reported to be rare in cardiac deep learning generally, with most pipelines trained and evaluated on curated data that under-represents motion artefact, low contrast, and rare pathology (PMC13007597; arXiv 2609.17068), which reframes the absence of an OOD-aware QC layer here as a documented, acknowledged gap in the wider field, not a shortcut specific to this project.

Because CGPS has no manually drawn coronary segmentation, essentially the entire QC apparatus this repository already has, Dice, Hausdorff-95, `cl_dice`, centerline mean distance, Betti error, is unusable as written, since every metric function in `utils/metrics.py` takes a ground-truth mask or centerline as a required argument. What remains usable without any ground truth falls into four categories, and this plan uses all four rather than inventing a fifth. First, the two postprocessing filters CAS-Net already runs unconditionally, component count and size filtering, are a free-standing sanity signal on their own: since the mask is designed to leave behind exactly two components (one per coronary tree), the count and size distribution of components before and after the size filter is directly informative on CGPS with no code change beyond adding a log line to record what the filter drops, which does not exist yet. Second, the underlying Betti-number primitive, b_0 (connected components) and b_1 (independent loops) computed on the prediction mask alone via `scipy.ndimage.label` and `skimage.measure.euler_number`, exactly as `utils/metrics.py`'s `_betti_numbers()` already computes internally (`utils/metrics.py:4-17`), is a usable ground-truth-free red flag: an unusually high b_0 signals fragmentation beyond the expected two trees, and any nonzero b_1 signals a loop, which is anatomically implausible for a coronary tree and a strong indicator of a segmentation artefact; this primitive is not currently exposed as a standalone metric outside the paired-error framing `evaluate.py` uses, so exposing it as its own small script is new, minimal code, not a research problem. Third, `utils/display_in_slicer.py`'s existing "segments" mode, already discussed above for the left/right substitution, doubles as the general visual-review tool for spot-checking any CGPS prediction and its derived centerline in 3-D, with no code change beyond repointing its directory constants. Fourth, `inference.py --computational_analysis` gives a run-health check, parameter count, per-scan timing, peak GPU memory, useful for confirming the job behaved plausibly on CGPS's likely different scan dimensions, though it is a runtime sanity check, not an output-quality check, and should not be mistaken for one.

Two further, genuinely useful QC signals require more than a config change and are explicitly proposed as new, minimal work rather than assumed to exist. Reverse Classification Accuracy is a documented method for per-case quality estimation with literally no ground truth at deployment: it trains a one-off classifier from a single test image plus its own predicted segmentation as pseudo-ground-truth, then checks whether that classifier can also segment a small, separately and manually verified reference set well; applied to UK Biobank cardiac MRI it reached 99 percent accuracy distinguishing good from poor segmentations across a 400-case experiment (PMC6416857). Its precondition for CGPS is a small manually segmented reference subset from CGPS itself, and

whether the thesis budget or clinical collaborators can produce even a handful of such cases is an open design question for supervisors, not something this plan can resolve unilaterally. Test-time-augmentation variance is a second candidate, cheaper mechanically since CAS-Net’s inference already runs four mirror-flip forward passes per scan and averages their sigmoid outputs; recording the per-voxel or per-vessel variance across those four passes, rather than only their mean, gives a model-agnostic, no-retraining-needed uncertainty proxy directly applicable to a frozen network ([PMC13514988](#)), but one recent paper explicitly warns that test-time augmentation can hurt rather than help in some medical-imaging settings ([arXiv 2604.09697](#)), so this plan treats TTA variance as a proxy to validate against the small manual-review sample above, not as a signal to trust by default; capturing it requires a small addition to `inference.py` to accumulate per-combo probabilities before averaging, which is not currently saved. Finally, a lightweight, code-light comparison of image-domain statistics, HU histogram in the vessel region of interest, slice thickness, reconstruction kernel, contrast timing, between ImageCAS-X and CGPS is worth running before any of the above, precisely because the cited generalisation study found simple contrast and sharpness statistics predictive of the resulting Dice drop; this needs only a short comparison script reading existing DICOM/NIfTI headers, not a learned model.

What must happen before this plan can run

The following steps are ordered roughly by dependency, and none of them can be skipped by writing around the gap; each is either a recovery from a branch that already has the code, or new code this plan requires that has never existed anywhere in the project’s history.

1. Write a CGPS-specific inference config (`configs/cas_net_cgps.json` or similar), pointing `data.volume_dir/data.volume_suffix` at wherever CGPS scans land on `/work3`, and decide whether CGPS volumes get an offline 0.5 mm resample-to-disk pass mirroring `utils/offline_resample_images_to_disk.py`, or run the Resample step at load time by omitting "resample" from `preprocessing.steps`; no such config exists in the repository today.
2. Recover the mask-to-centerline extractor from week3: `git show week3:topology/skeleton.py` > `topology/skeleton.py`, plus its driver and validation scripts, `git show week3:scripts/build_predicted_centerlines.py` > `scripts/build_predicted_centerlines.py` and the `validate_extraction.py` counterpart, since these were named together in the removal commit as the working entry point around `skeleton.py`.
3. Replace `skeleton.py`’s two oracle providers with the non-GT substitutes described above: drop `oracle_labels()` in favour of connected-component-based left/right splitting (already implicit in CAS-Net’s postprocessing design), and replace GT-ostium matching with a TotalSegmentator-derived coarse aortic-root proximity rule. This is new code; nothing in any branch checked implements a non-oracle version of either provider.
4. Recover only the `keep_largest` ("keep2") mode of `postprocessing/qiu/reconnect.py` from week3, and a corresponding postprocessing config akin to week3’s `configs/cas_net_keep2.json`, wiring it into the CGPS postprocessing chain ahead of the mask-to-centerline step; the classifier, walk, and skeleton modules used only by the full "qiu" mode are explicitly not needed and this plan does not recover them.
5. Fix the regional κ_a denominator bug in `tortuosity/research/case_study/regional.py` (start every proximal window 5 mm from its junction, or divide by $\ell/2$ times the attributed vertex count) before computing any regional descriptor; this is not required for the whole-vessel κ_a/TI pair, which does not use that code path.

6. Reconcile the π -factor mismatch between `tortuosity/merged.py`'s `scc_density()` and the $\kappa_a = \sum_j \theta_j / L$ convention this plan and every cited normal-range number use, and fix the vessel definition to `tortuosity/vessels.py`'s longest-geodesic-path convention rather than `topology.graph.Vessel`'s, before computing anything that will be quoted in the thesis.
7. Consolidate the presently scattered, partially overlapping descriptor scripts under `tortuosity/research/` into one dated, pre-registered descriptor module under `tortuosity/`, since no single frozen version currently exists and the project's own audit names this consolidation as the single most urgent outstanding gap before any number can be cited.
8. Obtain the CGPS covariate table (age, sex, plus whatever additional biomarker columns exist) with confirmed per-patient, per-vessel correspondence to the CAS-Net-plus-recovered-skeleton output, and confirm with supervisors whether any small manually segmented CGPS reference subset can be produced to support a Reverse-Classification-Accuracy-style QC check; both are logistics and access questions, not code gaps.
9. Write a CGPS-specific job pair analogous to `jobs/infer_eval_cas_net.sh` that runs `inference.py` only, deliberately never invoking `evaluate.py`, and adds the small standalone scripts for pre-filter component-count logging, the standalone Betti-number check, and the image-domain-statistics comparison described above.
10. Date and commit a short pre-registration document before any CGPS prediction is examined for its descriptor values: the primary descriptor (κ_a per named vessel), the primary confirmatory test (the age-and-sex LMM as specified), the biomarker-panel extension rule and its Benjamini-Hochberg correction, and the locked smoothing/chord operating point ($\sigma = 0$ extra smoothing, $\ell = 5$ mm chord, 8 mm fallback), so that this analysis does not repeat the "no pre-registered hypothesis" failure mode the project's own readiness audit already flagged as blocking for the ImageCAS-X-only protocol.

Every deviation from the trained-and-evaluated ImageCAS-X path

For a fairness claim to mean anything, every place this plan departs from the settings CAS-Net was actually trained and evaluated under on ImageCAS-X has to be stated, not discovered later. CGPS is out-of-distribution relative to the ImageCAS-X training cohort in scanner, protocol, and referral population, and the network is never retrained, fine-tuned, or given a single gradient step on CGPS data; every prediction comes from the same strictly loaded `cas_net.pt` state dict used for the delivered pretrained ImageCAS-X evaluation. CGPS has no ground-truth mask or centerline at all, so `evaluate.py`'s entire metric suite, Dice, HD95, `cl_dice`, centerline mean distance, Betti error, cannot run and is replaced by the ground-truth-free QC layer described above, which has never been validated against a manual reference on this project and is not a substitute for the metrics it replaces, only the best available proxy. The postprocessing chain adds a deterministic "keep2" component-cleanup step ahead of centerline extraction that the delivered, evaluated ImageCAS-X pipeline does not use by default, a deviation adopted specifically because CGPS is expected to produce more spurious fragments than ImageCAS-X's own test split did. The centerline extractor itself, once recovered from `week3`, runs with two heuristic, non-ground-truth-dependent providers, in place of the oracle providers `skeleton.py` uses everywhere else in the project, and those heuristics have not been validated against any ground truth, on CGPS or ImageCAS-X, because none exists for either. The tortuosity descriptors are computed at an operating point, no additional smoothing and a 5 mm chord, that was selected using development-distribution behaviour observed exclusively on ImageCAS-X's own training and test scans, and transplanting that operating point onto CGPS without re-running any reliability check there is itself an assumption, mitigated only by the fact that the case study measured κ_a 's ranking to be stable to smoothing choice in the first place. The

statistical model is a general biostatistics LMM structure, not a coronary-tortuosity-specific validated equation, since no such published equation exists in the reviewed literature, and its variance components will be estimated fresh from CGPS itself rather than carried over from anywhere. And, stated once more because it is easy to state quietly and misleadingly otherwise, no part of this plan reproduces an existing CGPS-specific normal range or effect size for age or sex on coronary tortuosity; every such number this plan eventually produces is the first one, on this cohort, for these descriptors, and should be written up in the thesis as exactly that.

What this plan changes about how the thesis should read

The honest framing this plan forces is that CGPS is not a validation set for CAS-Net, it is a deployment, and the entire value of running it comes from what can be learned with no ground truth in the loop at any stage, from segmentation through to the age/sex association test. That reframing is why the plan spends as much space on QC-without-ground-truth and on naming which oracle dependencies have to be replaced as it spends on the statistical model itself: a mixed-effects model fit on a biased or silently mis-labelled centerline tree is not a modest limitation, it is a wrong answer stated with a confidence interval. The one genuinely new capability this plan adds beyond what the project already has is the discipline of treating every one of the four QC signals, component count, Betti primitive, visual spot-check, and domain-shift statistics, as a pre-registered gate rather than an after-the-fact excuse, which is the same discipline the project's own readiness audit already demanded of the ImageCAS-X-only tortuosity protocol and found missing there too.